

Part I: Divide and Conquer

Lecture 4: Randomized Partition and Randomized Selection

Computer Science and Technology
United International College

Objective and Outline

Objective: Show two examples that use both (1) divide and conquer and (2) randomization

Reference: Chapter 7, 9 of CLRS

Outline:

1 Partition

- Basic partition
- Randomized partition and randomized quicksort
- Analysis of the randomized quicksort

2 Selection

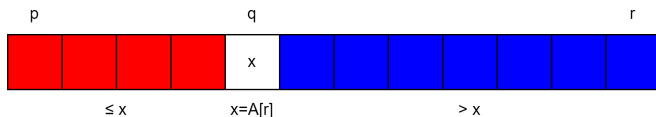
- The selection problem
- First solution: Selection by sorting
- A divide-and-conquer algorithm
- Analysis of the divide-and-conquer algorithm

Definition 1 (Partition)

- **Input:** An array of **distinct** numbers
- **Output:** Rearrange the array $A[p \cdots r]$ into two (possibly empty) subarrays $A[p \cdots q - 1]$ and $A[q + 1 \cdots r]$ such that

$$A[u] \leq A[q] \leq A[v]$$

for any $p \leq u \leq q - 1$ and $q + 1 \leq v \leq r$.

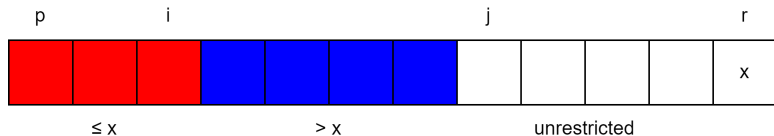


Algorithm 1 *QuickSORT*(A)

- 1: **if** $A.length > 1$ **then**
- 2: $Partition(A)$ into $A_1 = A[p \cdots q - 1]$ and $A_2 = A[q + 1 \cdots r]$
- 3: $QuickSORT(A_1)$ and $QuickSORT(A_2)$
- 4: **end if**

The Idea of Partition(A, p, r)

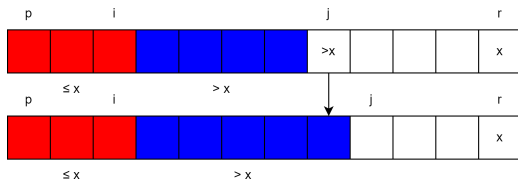
Use $A[r]$ as the **pivot**, and grow partition from left to right



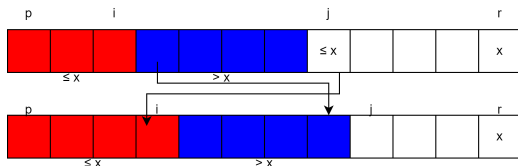
- 1 Initially, $(i, j) = (p - 1, p)$.
- 2 Increase j by 1 each time to find a place for $A[j]$. At the same time increase i when necessary.
- 3 Stops when $j = r$.

One Iteration of the Procedure Partition

Increase j by 1 each time to find a place for $A[j]$. At the same time increase i when necessary



- 1 If $A[j] > x$, only increase j by 1.



- 2 If $A[j] \leq x$, increase i by 1, $SWAP(A[i], A[j])$, and increase j by 1.

Example: The Operation of Partition(A, p, r)

i	p, j						r	
	2	8	7	1	3	5	6	4

(1)

p, i	j						r
2	8	7	1	3	5	6	4

(2)

p, i	j			r			
2	8	7	1	3	5	6	4

(3)

p, i	j			r			
2	8	7	1	3	5	6	4

(4)

p	i			j			r
2	1	7	8	3	5	6	4

(5)

p		i		j		r	
2	1	3	8	7	5	6	4

(6)

p		i			j	r	
2	1	3	8	7	5	6	4

(7)

p		i				j, r	
2	1	3	8	7	5	6	4

(8)

p		i				j, r	
2	1	3	4	7	5	6	8

(9)

The Partition(A, p, r) Algorithm

Algorithm 2 Partition(A, p, r)

```
1:  $x = A[r]$                                 //  $A[r]$  is the pivot element
2:  $i = p - 1$ 
3: for  $j = p$  to  $r - 1$  do
4:   if  $A[j] \leq x$  then
5:      $i = i + 1$ 
6:     exchange  $A[i]$  and  $A[j]$ 
7:   end if
8: end for
9: exchange  $A[i + 1]$  and  $A[r]$            // put pivot in position
10: return  $i + 1$                         //  $q = i + 1$ 
```

Running Time of Partition(A, p, r)

Algorithm 3 Partition(A, p, r)

```
1:  $x = A[r]$ 
2:  $i = p - 1$ 
3: for  $j = p$  to  $r - 1$  do           //  $r - p$  times
4:     if  $A[j] \leq x$  then
5:          $i = i + 1$ 
6:         exchange  $A[i]$  and  $A[j]$ 
7:     end if
8: end for
9: exchange  $A[i + 1]$  and  $A[r]$ 
10: return  $i + 1$ 
```

Total: $(r - p)$

Running time is $\Theta(r - p)$

- linear in the length of the array $A[p \cdots r]$

1 Partition

- Basic partition
- Randomized partition and randomized quicksort
- Analysis of the randomized quicksort

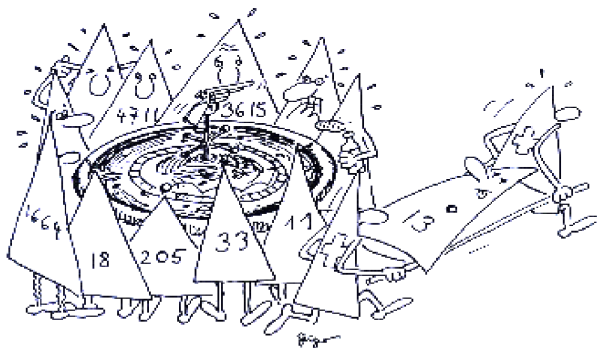
2 Selection

- The selection problem
- First solution: Selection by sorting
- A divide-and-conquer algorithm
- Analysis of the divide-and-conquer algorithm

Randomized-Partition(A, p, r)

Idea:

- In the algorithm 2 Partition(A, p, r), $A[r]$ is always used as the pivot x to partition the array $A[p \cdots r]$.
- In the algorithm 4 Randomized-Partition(A, p, r), we randomly choose an j , $p \leq j \leq r$, and use $A[j]$ as the pivot.



Randomized-Partition(A, p, r)

Let *random*(p, r) be a pseudorandom-number generator that returns a random number between p and r .

Algorithm 4 Randomized-Partition(A, p, r)

- 1: $j = \text{random}(p, r)$
 - 2: exchange $A[r]$ and $A[j]$
 - 3: Partition(A, p, r)
-

Randomized-Quicksort Algorithm

We make use of the Randomized-Partition idea to develop a new version of quicksort.

Algorithm 5 Randomized-Quicksort(A, p, r)

```
1: if  $p < r$  then  
2:    $q = \text{Randomized-Partition}(A, p, r)$   
3:   Randomized-Quicksort( $A, p, q - 1$ )  
4:   Randomized-Quicksort( $A, q + 1, r$ )  
5: end if
```

1 Partition

- Basic partition
- Randomized partition and randomized quicksort
- Analysis of the randomized quicksort

2 Selection

- The selection problem
- First solution: Selection by sorting
- A divide-and-conquer algorithm
- Analysis of the divide-and-conquer algorithm

Running Time of the Randomized-Quicksort

Measuring running time:

- The running time is dominated by the time spent in partition.
- The running time of the partition procedure can be measured by **the number of key comparisons**.

Let $T(n)$ be the running time on array of size n .

Recurrence: $T(n) = T(m) + T(n - m - 1) + \Theta(n)$

Worst Case: $T(n) = T(0) + T(n - 1) + \Theta(n)$

$$T(n) = \Theta(n^2)$$

What inputs give worst case performance?

- Whether performance is the worst is not determined by input.
- An important property of randomized algorithms.
- Worst case performance results if the random number generator always produces the worst choice.

There are two observations about key comparisons.

Observation 1

*When a pivot is selected, the pivot is compared with **every** other elements, then the elements are partitioned into two parts accordingly.*

Observation 2

*Elements in **different** partitions are **never** compared with each other in **all** operations.*

Average Running Time of Randomized-Quicksort

- Let A be the input array which is a permutation of the n distinct elements $z_1 < z_2 < \dots < z_n$,
- X be the total number of comparisons performed in **all** calls to randomized-partition, and
- X_{ij} be the number of comparisons between z_i and z_j .
 - X_{ij} can only be **0** or **1**.

$$\begin{aligned} E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n [P_r\{z_i \text{ is compared with } z_j\} \times 1 \\ &\quad + P_r\{z_i \text{ is not compared with } z_j\} \times 0] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n P_r\{z_i \text{ is compared with } z_j\} \end{aligned}$$

Key Observations on Finding $P_r\{z_i \text{ is compared with } z_j\}$

For $1 \leq i \leq j \leq n$, let $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$.

- Remember $z_i < z_{i+1} < \dots < z_j$.

Observation 3

Partition divides an array into three segments, left, pivot, and right.

Observation 4

When z_i and z_j are first placed in DIFFERENT segments of the array by partitioning, the pivot is one of the elements in Z_{ij} .

Observation 5

If the pivot is either z_i or z_j , z_i and z_j will be compared.

Observation 6

*If the pivot is any element in Z_{ij} other than z_i or z_j , z_i and z_j are **not** compare with each other in **all** randomized-partition calls.*

How to Find $P_r\{z_i \text{ is compared with } z_j\}$?

$$P_r\{z_i \text{ is compared with } z_j\}$$

$$= P_r\{z_i \text{ or } z_j \text{ is the first pivot chosen from } Z_{ij}\}$$

$$= P_r\{z_i \text{ is the first pivot chosen from } Z_{ij}\}$$

$$+ P_r\{z_j \text{ is the first pivot chosen from } Z_{ij}\}$$

$$= \frac{1}{j-i+1} + \frac{1}{j-i+1} = \frac{2}{j-i+1}$$

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n P_r\{z_i \text{ is compared with } z_j\} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^{n-1} \mathcal{O}(\lg n) = \mathcal{O}(n \lg n) \end{aligned}$$

NOTE: $\sum_{k=2}^n \frac{1}{k} \leq \log(n)$

Hence, the expected number of comparisons is $\mathcal{O}(n \lg n)$, which is the average running time of Randomized-Quicksort.

1 Partition

- Basic partition
- Randomized partition and randomized quicksort
- Analysis of the randomized quicksort

2 Selection

- The selection problem
- First solution: Selection by sorting
- A divide-and-conquer algorithm
- Analysis of the divide-and-conquer algorithm

Linear Time Selection

Definition (Selection Problem)

Given a sequence of numbers $\langle a_1, a_2, \dots, a_n \rangle$, and an integer i , $1 \leq i \leq n$, find the **i th smallest element**. When $i = \lceil n/2 \rceil$, it is called the **median problem**.

Example 2

Given $\langle 1, 8, 23, 10, 19, 33, 100 \rangle$, the 4th smallest element is 19.

Question

How do you solve this problem?

1 Partition

- Basic partition
- Randomized partition and randomized quicksort
- Analysis of the randomized quicksort

2 Selection

- The selection problem
- **First solution: Selection by sorting**
- A divide-and-conquer algorithm
- Analysis of the divide-and-conquer algorithm

First Solution: Selection by Sorting

The selection problem can easily be solved by sorting.

- 1 Sort the array in the ascending order ($\mathcal{O}(n \log n)$).
- 2 Return the i -th element of the sorted array ($\mathcal{O}(1)$).

The complexity of this method is $\mathcal{O}(n \log n)$.

Question

Can we do better?

Answer: YES, but we need to recall $\text{Partition}(A, p, r)$ used in Quicksort!

1 Partition

- Basic partition
- Randomized partition and randomized quicksort
- Analysis of the randomized quicksort

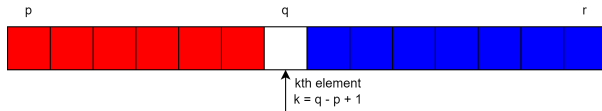
2 Selection

- The selection problem
- First solution: Selection by sorting
- **A divide-and-conquer algorithm**
- Analysis of the divide-and-conquer algorithm

Randomized-Select(A, p, r, i), $1 \leq i \leq r - p + 1$

Problem: Select the i -th smallest element in $A[p \cdots r]$, where $1 \leq i \leq r - p + 1$.

Solution: Apply Randomized-Partition(A, p, r).



- 1 If $i = k$,
 - pivot is the solution;
- 2 If $i < k$,
 - the i -th smallest element in $A[p \cdots r]$ must be the i th smallest element in $A[p \cdots q - 1]$; and
- 3 If $i > k$,
 - the i -th smallest element in $A[p \cdots r]$ must be the $(i - k)$ -th smallest element in $A[q + 1 \cdots r]$.

Randomized-Select(A, p, r, i), $1 \leq i \leq r - p + 1$

If necessary, **recursively** call the same procedure to the subarray.

Algorithm 6 Randomized-Select(A, p, r, i)

```
1: if  $p = r$  then
2:   return  $A[p]$ 
3: end if
4:  $q = \text{Randomized-Partition}(A, p, r)$ 
5:  $k = q - p + 1$ 
6: if  $i = k$  then
7:   return  $A[q]$            // the pivot is the answer
8: else if  $i < k$  then
9:   return Randomized-Select( $A, p, q - 1, i$ )
10: else  $i > k$ 
11:   return Randomized-Select( $A, q + 1, r, i - k$ )
12: end if
```

To find the i -th smallest element in $A[1 \cdots n]$, call
Randomized-Select($A, 1, n, i$).

1 Partition

- Basic partition
- Randomized partition and randomized quicksort
- Analysis of the randomized quicksort

2 Selection

- The selection problem
- First solution: Selection by sorting
- A divide-and-conquer algorithm
- Analysis of the divide-and-conquer algorithm

Running Time of Randomized-Select($A, 1, n, i$)

$T(n, i)$ is the **average** number of comparisons made by Randomized-Select($A, 1, n, i$).

- $T(1, 1) = 0$
- For $n > 1$,

$$\begin{aligned} T(n, i) = & (n - 1) && \text{initial partition} \\ & + \frac{1}{n} \left(\sum_{k=1}^{i-1} T(n - k, i - k) \right) && \text{recursion, } k < i \\ & + \sum_{k=i+1}^n T(k - 1, i) && \text{recursion, } k > i \end{aligned}$$

We will prove by induction on n that

$$T(n, i) < 4n, \text{ for all } n \geq 1 \text{ and } 1 \leq i \leq n.$$

Running Time of Randomized-Select($A, 1, n, i$)

Proof.

Base case: $T(1, 1) = 0 < 4 \times 1$

Induction (strong): Assume that $T(m, j) < 4m$ for all $m < n$ and $1 \leq j \leq m$. Then,

$$\begin{aligned} T(n, i) &= (n-1) + \frac{1}{n} \left(\sum_{k=1}^{i-1} T(n-k, i-k) + \sum_{k=i+1}^n T(k-1, i) \right) \\ &< (n-1) + \frac{1}{n} \left(\sum_{k=1}^{i-1} 4(n-k) + \sum_{k=i+1}^n 4(k-1) \right) \\ &= (n-1) + \frac{1}{n} \left(4 \frac{(2n-i)(i-1)}{2} + 4 \frac{(i+n-1)(n-i)}{2} \right) \\ &= (n-1) + \frac{1}{n} \underbrace{(2n^2 - 6n + (4n+4)i - 4i^2)}_{f(i)} \end{aligned}$$

Running Time of Randomized-Select($A, 1, n, i$)

Proof.(Con't)

$$T(n, i) < (n - 1) + \frac{1}{n}f(i)$$

where

$$f(x) = 2n^2 - 6n + (4n + 4)x - 4x^2$$

To find the maximum value of $f(x)$,

$$f'(x) = (4n + 4) - 8x = 0 \text{ for } x = \frac{n + 1}{2}$$

$$f''(x) = -8 < 0$$

Hence,

$$f(x) \leq f\left(\frac{n + 1}{2}\right) = 3n^2 - 4n + 1, \text{ for all } x$$

Therefore,

$$T(n, i) < (n - 1) + 3n - 4 + \frac{1}{n} < 4n$$

Running Time of Randomized-Select($A, 1, n, i$)

Since $T(n, i) \geq n - 1$, we have in particular that

$$T(n, i) = \Theta(n)$$

Worst Case:

$$T(n, i) = (n - 1) + T(n - 1, i) = \Theta(n^2)$$

Randomized Quicksort vs Randomized Selection

Question

Why does Randomized Selection take $\Theta(n)$ time while Randomized Quicksort takes $\mathcal{O}(n \log n)$ time?

Answer:

- Randomized Selection needs to work on only **1** of the two subproblems.
- Randomize Quicksort needs to work on **both** of the two subproblems.